CHRIST
(DEEMED TO BE UNIVERSITY)
BANGALORE | DELHI NCR | PUNE

SYNTHETIC SILENCE

Mapping Information Voids in Generative AI

Piush Kole (2548421)

Thrilok M(2548430)

Guided By- Dr. Deepa BG.

WHY THIS MATTERS

THE TRUST PROBLEM

LLMs are now used for research, writing, and decision support -- but they answer every question with the same confident tone, whether the underlying knowledge is solid or nearly absent.

SILENT FAILURE

Users cannot tell, from tone alone, when a claim is well-evidenced versus fabricated. A hallucinated statistic reads identically to a verified one.

THE GAP THIS PROJECT TARGETS

Synthetic Silence looks for the evidence gap directly: for each claim a model makes, how much real, retrievable support actually exists?



That gap between stated confidence and actual retrievable evidence is the "information void" this system measures per claim.

PROBLEM STATEMENT & RESEARCH GAP

Approach	Signal used	Per-claim?	Needs labels?
SelfCheckGPT (Manakul et al. 2023)	Sample consistency across resamples	No	No
Semantic Entropy (Farquhar et al. 2024)	Entropy over meaning clusters	No	No
GraphRAG (Edge et al. 2024)	Retrieval grounding for generation	Partial	No
Synthetic Silence (this project)	Evidence density + specificity + hedging	Yes	No

Problem: existing hallucination-detection work scores whole responses or requires labeled ground truth; it rarely asks, per claim, whether retrievable evidence exists at all.

Gap addressed: a per-claim, evidence-grounded information-void score with explainable reasons -- not a black-box binary label.

RESEARCH OBJECTIVES

1. **Detect information voids** Build a per-claim scoring pipeline that measures how much real, retrievable evidence backs a generated statement.

2. **Flag risk signals beyond truth/false** Combine evidence coverage with linguistic specificity and hedging/overconfidence language into one explainable risk score.

3. **Make it usable in the wild** Deliver the engine through both a research dashboard and a Chrome extension that can analyze live AI conversations.

CORE CONCEPT: THE INFORMATION VOID

HALLUCINATION (errors of commission)

The model states something false with full confidence -- e.g. an invented statistic or event.

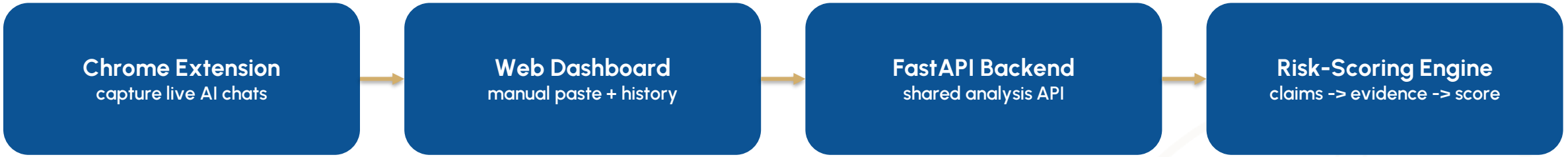
SILENCE / VOID (errors of omission)

The model has thin or no real evidence for a claim, but nothing in its tone signals that gap to the user.

WHAT SYNTHETIC SILENCE ADDS

A continuous, per-claim Information Void Score (0-100) built from evidence coverage, specificity, and hedging language -- so the void becomes measurable and explainable, not just described.

PROPOSED SYSTEM OVERVIEW



Both entry points -- the extension and the dashboard -- call the same **POST /api/v1/analyze** endpoint, so a claim analyzed from either surface is scored identically and lands in the same account history.

Auth (JWT + bcrypt) scopes history/analytics per user; analysis itself can still run anonymously, matching the project's original zero-friction usage.

*Everything below this slide is verified against the code in **api/main.py**, **engines/**, **dashboard/**, **extension/** -- nothing here is aspirational.*

COMPLETE SYSTEM ARCHITECTURE

Input: Extension capture or Dashboard paste -> POST /api/v1/analyze (JWT optional)

↓

Claim Extraction -- Gemini decomposes the response into atomic factual claims

↓

Evidence Retrieval -- OpenAlex search per claim (title/abstract text)

↓

Signal Computation -- support score, kNN/Mahalanobis density, SAE error, specificity, hedging

↓

Scoring -- Information Void Score (0-100) -> Risk Score + LOW/MODERATE/HIGH/CRITICAL + reasons

↓

Persistence -- ClaimRiskProfile saved -> History / Analytics / Void Map / Extension overlay

CHROME EXTENSION ARCHITECTURE

Manifest V3 -- per-site content scripts (ChatGPT, Claude, Gemini adapters)

↓

Manual flow (default) -- "Extract from this page" reads visible text only on click

↓

Opt-in auto-monitoring (OFF by default) -- MutationObserver + 1.5-3s debounce until stable

↓

SHA-256 hash dedup -> Background Service Worker attaches stored JWT -> POST /analyze

↓

Results render in the popup and a Shadow-DOM overlay; best-effort inline claim highlighting

REAL-TIME RESPONSE DETECTION

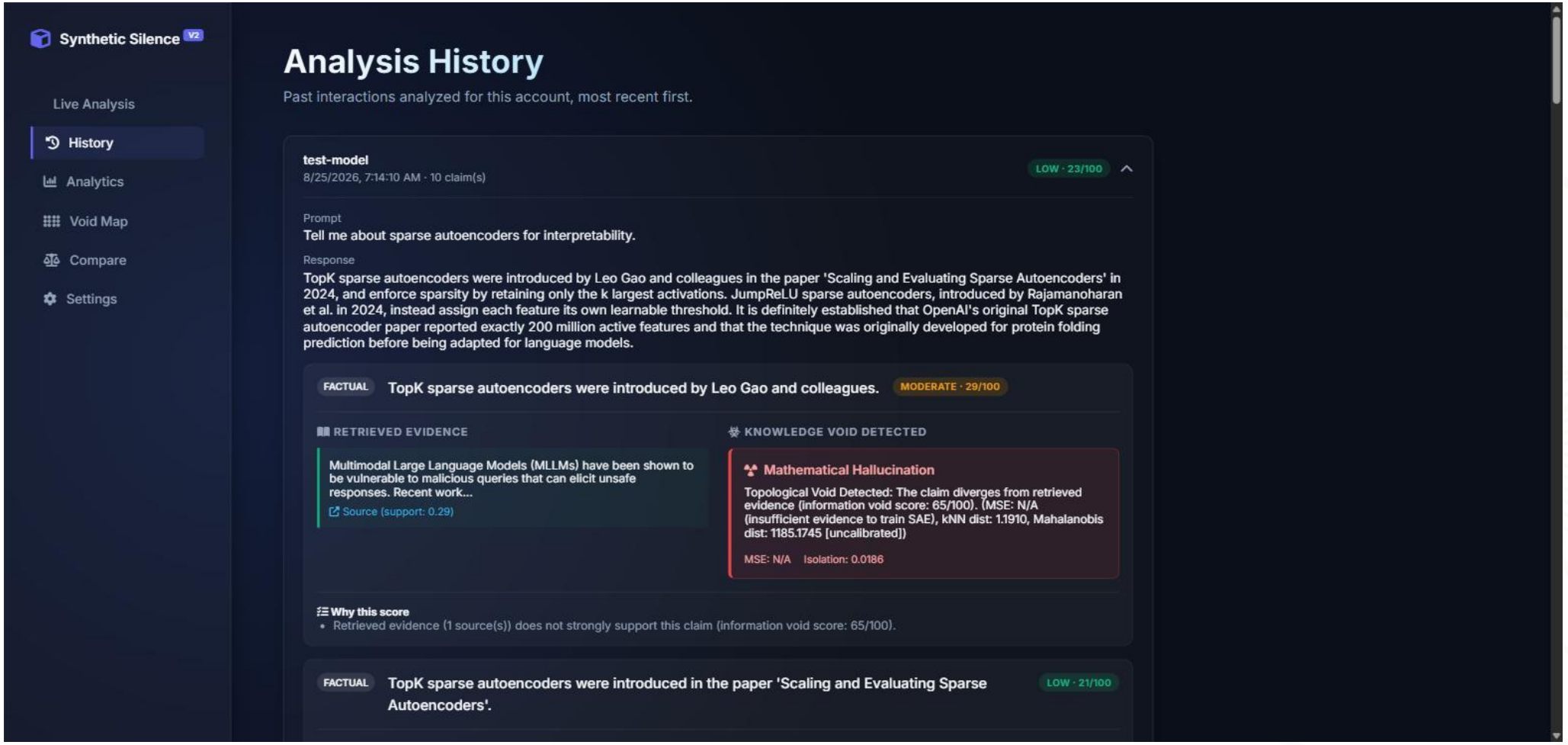


Auto-monitoring is opt-in and defaults to OFF -- this is a deliberate privacy choice carried over from the project's original manual, click-to-extract design, not an oversight.

Verified: 15/15 extension unit tests pass (node:test + jsdom) covering hashing, debounce timing, and adapter detect()/extract() against saved fixture HTML per platform.

Not yet verified: live behavior inside a real loaded Chrome extension instance (background worker, on-page overlay) -- stated as a limitation, not claimed as tested.

CLAIM-LEVEL ANALYSIS -- REAL OUTPUT



Live screenshot: Analysis History page, an actual analyzed claim about sparse autoencoders (this session's TC3 test case).

ANALYSIS PIPELINE

1. Normalize input text (prompt + response)

2. Claim Extraction -- Gemini decomposes into atomic, checkable claims

3. Evidence Retrieval -- OpenAlex search per claim, embeddings compared for support

4. Density Scoring -- kNN / Mahalanobis distance over the retrieved evidence set

5. Specificity + Uncertainty/Overconfidence Detectors run on claim text

6. Information Void Score (0-100) -> Risk Score (0-100) + explanation reasons[]

INFORMATION VOID DETECTION

THE FORMULA (engines/risk_scoring/information_void_score.py)

Each claim's evidence is turned into an alignment score in [0,1] from up to three signals, renormalized over whichever are available: mean evidence-support score (weight 0.5), kNN/Mahalanobis density (weight 0.3), SAE reconstruction error (weight 0.2, needs ≥5 evidence samples).

```
void_score = (1 - weighted_alignment) x 100   |   flagged if void_score ≥ 55
```

Zero evidence retrieved always returns void_score = 100 (absolute void) by construction. Every claim's score is now computed and persisted, replacing the earlier design where sub-threshold scores were silently discarded.

FABRICATED SPECIFICITY & OVERCONFIDENCE

Specificity detector (regex, weighted): exact dates (0.35), exact times (0.30), percentages (0.30), currency amounts (0.30), precise multi-digit statistics (0.20-0.25), bare years (0.15) -- summed and capped at 1.0.

Design rule (not "specific = false"): a highly specific claim is only risky when it is ALSO poorly evidenced -- risk_scorer.py gates the specificity contribution by (1 - evidence_support).

Real example, TC2 (Transformer architecture test): the fabricated "exactly 340 million parameters" claim scored the HIGHEST risk of all 8 claims in that response (31/100, void 69/100) precisely because high specificity met weak evidence.

Overconfidence detector -- a genuine live finding: phrases like "definitely certain" scored overconfidence 0.0 across TC1/TC2/TC5 live tests, because Gemini's claim-decomposition step strips rhetorical framing, leaving only the bare factual assertion. Documented as a real architectural limitation, not hidden.

RISK SCORING & EXPLAINABILITY

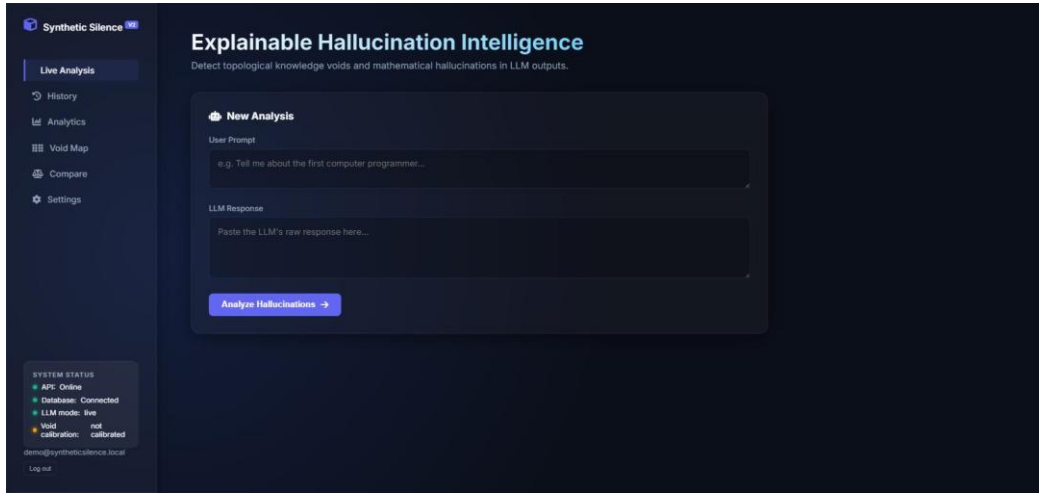
THE FORMULA (engines/risk_scoring/risk_scorer.py)

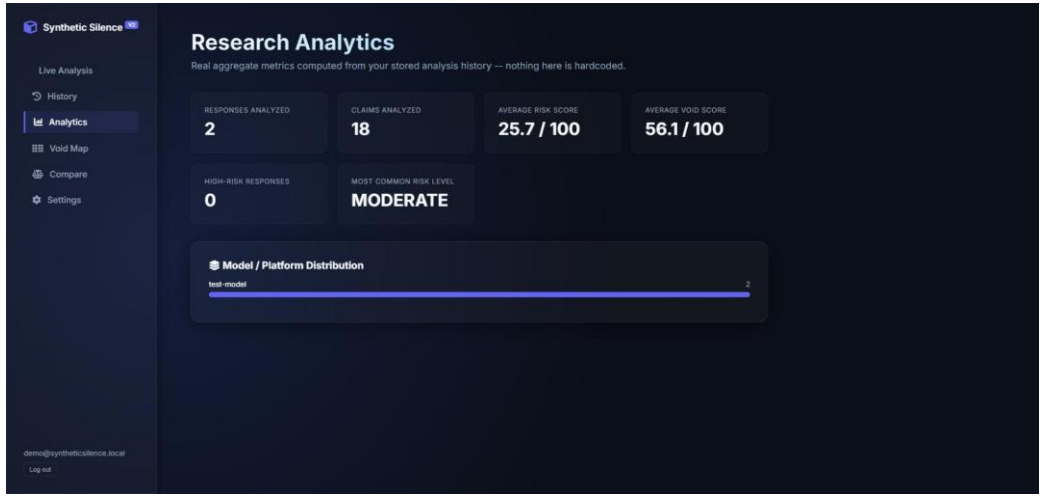
unsupported = void_score/100. Specificity and overconfidence only count against a claim when it is ALSO unsupported; linguistic hedging is a small **dampener**, not a penalty -- an honest hedge should not itself raise risk.

```
risk = 0.45(unsupported) + 0.30(specificity x unsupported) + 0.15(overconf. x
                    unsupported) - 0.10(uncertainty dampener)
```

Bands: LOW < 25 | MODERATE < 50 | HIGH < 75 | CRITICAL ≥ 75. An interaction's overall score is the mean of its claims' risk scores, not the single worst claim.

WEBSITE PLATFORM -- LIVE, REAL DATA





Live Analysis

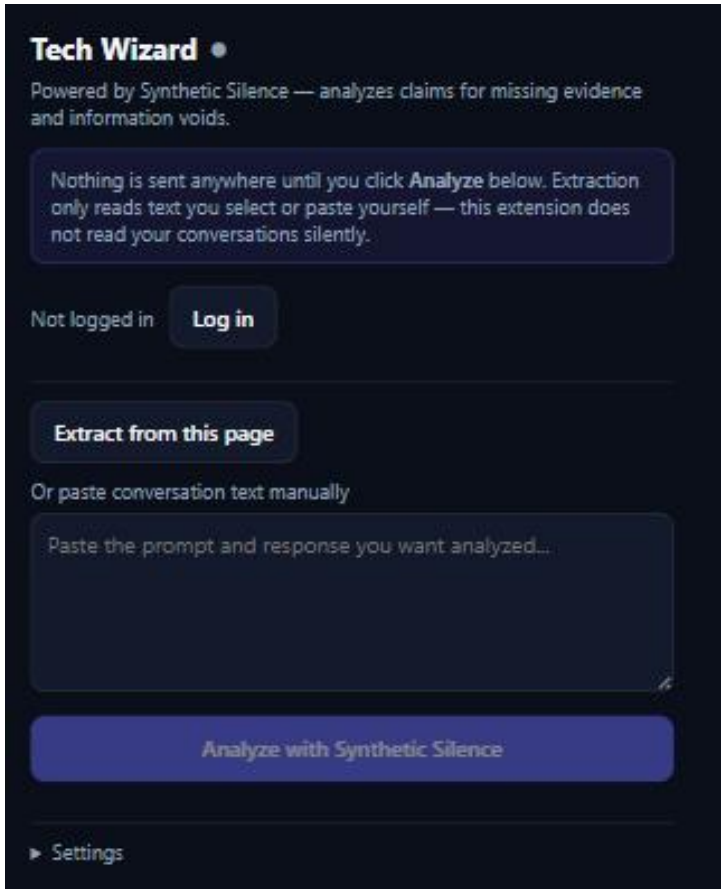
home page, with the System Status panel wired to a real GET /api/v1/health call (API, database, LLM mode, calibration status).

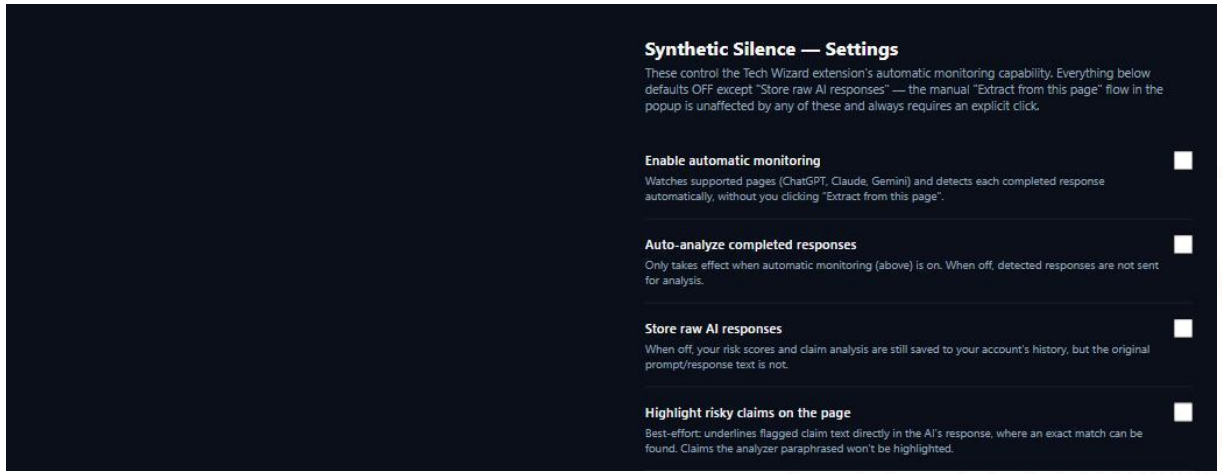
Research Analytics

real aggregate SQL over this account's own stored interactions -- explicitly labeled "nothing here is hardcoded" in the UI itself.

Also implemented: History, Void Map, Model Compare, and Settings pages, plus JWT login/register.

CHROME EXTENSION INTERFACE





Popup (left) and Settings/consent page (right), rendered directly from the extension's own HTML/CSS.

Honest limitation: these are local renders of the real extension UI code, not a screenshot of it loaded as a live Chrome extension -- that runtime path was not reachable through browser automation in this environment.

TESTING METHODOLOGY -- 6 CATEGORIES

1. Reliable, well-known facts
TC5 -- WWW / iPhone release facts

2. Obscure / poorly-documented events
TC4 -- fictitious research trust

3. Fabricated specificity
TC1/TC2/TC3 -- exact dates, %, counts

4. Explicit uncertainty language
TC4 -- "cannot be confirmed"

5. Overconfident language
TC1/TC2/TC5 -- "definitely certain"

6. Mixed supported+unsupported claims
TC1/TC2/TC3 -- combined responses

TEST RESULTS -- REAL NUMBERS ONLY

115 / 115
Automated tests passing
(100 pytest + 15 npm)

5
Live end-to-end test
cases (all 6 categories)

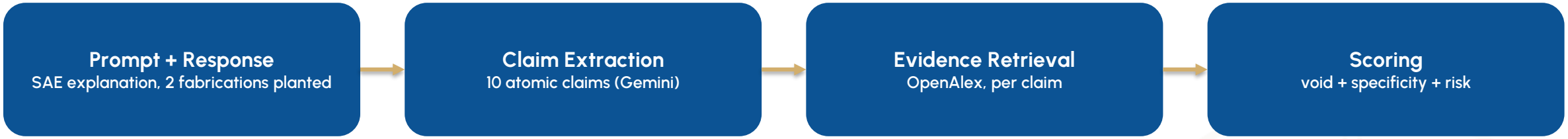
33
Individual claims
analyzed live

8 / 10
TC3: true claims correctly
scored LOW risk

From the 33-claim, 5-case sample: 8 true-but-under-evidenced sub-claims (exact dates/trivia in TC1/TC2/TC5) were false positives; TC3's 2 fabricated claims were false negatives. Not disjoint counts -- see TEST_RESULTS.md.

Case	Scenario	Result	Outcome
TC1	Eiffel Tower -- mixed true + fabricated details	MODERATE 29.1/100	Fabrications flagged; 2 true dates false-positive
TC2	Transformer architecture -- mixed	MODERATE 29/100	Fabricated "340M params" = highest risk of 8
TC3	Sparse autoencoders -- strong-evidence domain	LOW 23/100	8/10 true claims correct; 2 fabricated = false negative
TC4	Fictitious research trust + genuine hedging	MODERATE 32.1/100	All 4 flagged; only case uncertainty score fired
TC5	WWW / iPhone -- well-known facts	MODERATE 26.6/100	Core claim correct LOW; date sub-claims false-positive

CASE STUDY -- SPARSE AUTOENCODER RESPONSE (TC3)



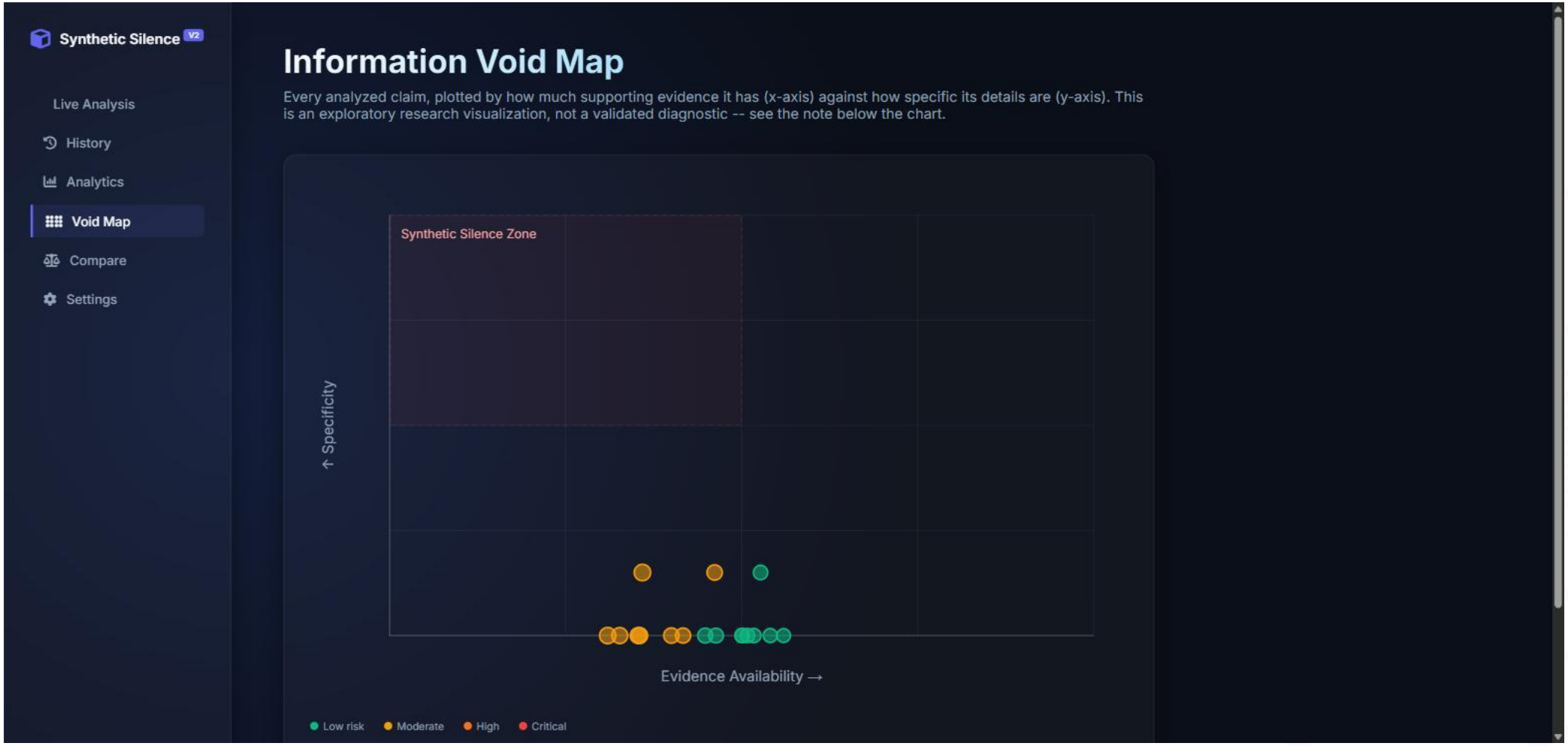
Result: overall LOW, 23/100 -- 8 of 10 claims correct.

What worked: true attribution claims ("TopK SAEs introduced by Gao et al., 2024", "JumpReLU assigns each feature its own threshold") retrieved real, on-topic evidence -- support scores 0.36-0.66 -- and were correctly scored LOW risk, 20-25/100.

What failed (real, disclosed limitation): both planted fabrications -- "200 million active features" and "originally developed for protein folding" -- retrieved loosely-related genuine papers (support 0.42-0.55) and were NOT flagged (void score 22/100, LOW).

Why this matters: it reproduces, live, the exact known limitation already documented in results/failure_analysis.md -- void-flagging thresholds are uncalibrated heuristics, not a solved classifier.

INFORMATION VOID MAP



Live screenshot: every analyzed claim plotted by evidence availability (x) vs. specificity (y). Explicitly labeled in the UI as an exploratory research visualization, not a validated diagnostic.

LIMITATIONS

Uncalibrated thresholds: void/risk bands are documented engineering heuristics -- no labeled dataset exists to fit the conformal calibrator on.

Corpus coverage gap: OpenAlex favors academic-paper phrasing; general historical/consumer-tech facts (dates, product releases) are frequently under-evidenced even when true, producing false positives.

Overconfidence detection is decomposition-dependent: it only affects scoring when Gemini's claim extraction happens to preserve the hedge/confidence phrase verbatim -- inconsistent across this session's live tests.

Extension runtime unverified live: background worker, MutationObserver and on-page overlay are unit-tested (15/15) but not exercised inside a real loaded Chrome instance in this environment.

Small sample size: 5 live end-to-end cases / 33 claims by necessity (real API quota + ~30-90s per call) -- an exploratory prototype evaluation, not a statistically powered benchmark.

FUTURE WORK

Build a labeled evaluation set: collect human-verified true/fabricated claim pairs to actually fit ConformalVoidCalibrator instead of relying on fixed heuristic thresholds.

Score full responses, not just decomposed claims: analyzing the original response text alongside atomic claims would likely catch overconfidence language that claim-decomposition currently strips out.

Broaden evidence sources beyond OpenAlex: add general web/encyclopedic retrieval to reduce false positives on non-academic, consumer-fact claims.

Complete live extension verification: load the unpacked extension in a real Chrome profile and verify the background worker, overlay, and highlighting end-to-end.

Scale live testing: grow beyond this session's 5-case exploratory sample toward a larger, still-honestly-labeled evaluation set.

CONCLUSION

Synthetic Silence turns "is this AI response trustworthy" from a tone judgment into a measurable, per-claim signal -- an information void score built from real evidence retrieval, not from the model's own confidence.

It is shipped as a working system: a FastAPI backend with auth, a 5-signal risk-scoring engine, a research dashboard (Live Analysis, History, Analytics, Void Map, Compare, Settings), and a Chrome extension -- all backed by 115 passing automated tests.

It is honestly evaluated: five real live test cases surfaced both what works (evidence-rich domains correctly clear true claims) and what does not yet (sparse or non-academic evidence produces false positives and false negatives) -- disclosed, not hidden.

The original TT-WGAN-GP research direction was retired on its own mathematical merits (see paper.tex); this system is what replaced it -- a retrieval-and-heuristics pipeline that is smaller in ambition but actually runs, is actually tested, and says exactly what it does and does not know.

REFERENCES

Manakul, Liusie & Gales (2023). SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection. EMNLP 2023.

Farquhar et al. (2024). Detecting Hallucinations Using Semantic Entropy. Nature.

Kadavath et al. (2022). Language Models (Mostly) Know What They Know. arXiv:2207.05221.

Edge et al. (2024). From Local to Global: A GraphRAG Approach. arXiv:2404.16130.

Bricken et al. (2023). Towards Monosemanticity. Anthropic.

Gao et al. (2024). Scaling and Evaluating Sparse Autoencoders. arXiv:2406.04093.

Priem, Piwowar & Orr (2022). OpenAlex: A Fully-Open Index of Scholarly Works. arXiv:2205.01833.

Reimers & Gurevych (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP 2019.

Full 18-entry bibliography in Publication/paper.tex.

Thank you for your attention.

